

SimplifyAI - Comprehensive Codebase Analysis

Date: 2026-03-24

Purpose: Complete documentation of all files, functions, and their roles

Audience: Developers, maintainers, anyone needing to understand or modify the codebase

Table of Contents

1. Project Overview
 2. File Hierarchy
 3. Python Backend
 4. Electron Main Process
 5. React UI Layer
 6. Data Files
 7. Model Artifacts
 8. Data Flow
 9. How To Guide
-

1. Project Overview

What it is: A privacy-first text simplification tool for people with dyslexia, ADHD, and autism. Runs completely offline with no data sent to external servers.

Architecture: Python backend (T5 model + mode-specific post-processing) + Electron/React frontend with IPC communication.

Key Design Decisions:

- Offline-first (everything stays on device)
 - Mode-specific accessibility (dyslexia, ADHD, autism)
 - Rule-based post-processing (transparent, auditable)
 - CPU-only inference (no GPU required)
-

2. File Hierarchy

F:\chat_editor_mp\



■■■ Python Backend (Root Level)

- ■■■ simplify.py # CLI entry point
- ■■■ simplify_server.py # JSON stdin/stdout server for Electron
- ■■■ utils.py # Shared utilities
- ■■■ dyslexia_mode.py # Dyslexia post-processing
- ■■■ adhd_mode.py # ADHD post-processing
- ■■■ autism_mode.py # Autism post-processing
- ■■■ idiom_map.json # Idiom → literal meaning mappings
- ■■■ jargon_map.json # Jargon → plain language mappings

■

■■■ Electron App

- ■■■ electron-app/
 - ■■■ main.js # Electron main process
 - ■■■ preload.js # Context bridge for IPC
 - ■■■ index.html # HTML template
 - ■■■ package.json # npm config and dependencies
 - ■■■ vite.config.js # Vite bundler config
 - ■■■ eslint.config.js # ESLint rules
 - ■■■ src/
 - ■■■ main.jsx # React entry point
 - ■■■ App.jsx # Root component
 - ■■■ App.css # Component CSS (legacy)
 - ■■■ index.css # Global styles + Tailwind
 - ■■■ context/
 - ■■■ AppContext.jsx # Global application state
 - ■■■ ThemeContext.jsx # Theme switching
 - ■■■ components/
 - ■■■ layout/
 - ■■■ Header.jsx # App header with controls
 - ■■■ MetricsBar.jsx # Readability metrics display
 - ■■■ SettingsPanel.jsx # Accessibility settings modal
 - ■■■ editor/
 - ■■■ InputPanel.jsx # Text input area
 - ■■■ OutputPanel.jsx # Simplified output display
 - ■■■ modes/

- ■■■■ DyslexiaOutput.jsx # Dyslexia mode renderer

- ■■■■ ADHDFocusMode.jsx # ADHD focus navigator

- ■■■■ AutismOutput.jsx # Autism mode renderer

-

■■■ Model Files

- ■■■■ t5-simplifier/ # Fine-tuned T5-small model

- ■■■■ config.json

- ■■■■ generation_config.json

- ■■■■ model.safetensors

- ■■■■ tokenizer.json

- ■■■■ tokenizer_config.json

- ■■■■ checkpoint-2500/ # Training checkpoint

- ■■■■ checkpoint-5000/ # Training checkpoint

- ■■■■ t5-base/ # LoRA adapter experiments

- ■■■■ adapter_only/

-

■■■ Documentation

- ■■■■ docs/

- ■■■■ design.md

- ■■■■ project_explanation.md

- ■■■■ testing-instructions.md

- ■■■■ finetune_colab.ipynb

-

■■■ Plan Folder (New)

- ■■■■ plan/

- ■■■■ codebase-analysis.md # This file

- ■■■■ neurodivergent-improvements-design.md

-

■■■ UI Design Mockups

- ■■■■ stitch/

- ■■■■ [5 design folders with code.html and screen.png]

3. Python Backend

3.1 simplify.py - CLI Tool

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\simplify.py<<<ENDFONT>>>
<<<BOLD>>>Purpose:<<<ENDBOLD>>> Command-line interface for text simplification

Functions

Function	Signature	Description	Use Case
<<_select_by_task_complexity<<<ENDFONT>>>	<<(text: str) -> str<<<ENDFONT>>>	Heuristic: returns "t5-medium" if avg word length >6 or word count >200	Auto model selection based on text complexity
<<_select_by_device<<<ENDFONT>>>	<<() -> str<<<ENDFONT>>>	Uses psutil to check RAM; returns "t5-medium" if >4GB available	Auto model selection based on hardware
<<_select_model<<<ENDFONT>>>	<<(choice: str, text: str) -> str<<<ENDFONT>>>	Dispatcher for model selection strategies	CLI <<--model<<<ENDFONT>>> argument handling
<<_load_model<<<ENDFONT>>>	<<(model_name: str) -> tuple[model, tokenizer]<<<ENDFONT>>>	Load T5 model with LRU cache (max 3 models)	Efficient model reuse across requests
<<simplify_with_t5<<<ENDFONT>>>	<<(text: str, model_name: str) -> str<<<ENDFONT>>>	Sentence-by-sentence T5 inference with beam search	Core neural simplification
<<process_text<<<ENDFONT>>>	<<(text, mode, model_name, use_hyphenation) -> str<<<ENDFONT>>>	Orchestrates: T5 simplification → mode-specific formatter	Main processing pipeline
<<main<<<ENDFONT>>>	<<() -> None<<<ENDFONT>>>	CLI entry: argparse, file reading, processing, output	Running from terminal

How to Modify

<<<BOLD>>>Change model defaults:<<<ENDBOLD>>>

- Edit <<_select_by_task_complexity<<<ENDFONT>>> thresholds (line ~50)
- Modify <<_MAX_MODEL_CACHE_SIZE<<<ENDFONT>>> (line ~20) to cache more models

<<<BOLD>>>Adjust T5 generation parameters:<<<ENDBOLD>>>

Around line 115-120

```
outputs = model.generate(  
    inputs["input_ids"],  
    max_length=128, # Change max output length  
    num_beams=4, # Beam search width  
    length_penalty=0.8, # Lower = shorter outputs  
    no_repeat_ngram_size=3,  
    early_stopping=True,  
)
```

1.2 simplify_server.py - JSON Server

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\simplify_server.py<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Persistent server process for Electron IPC communication

Protocol

<<<BOLD>>>Input (stdin JSON line):<<<ENDBOLD>>>

```
{  
    "text": "Input text to simplify",  
    "mode": "dyslexia",  
    "useHyphenation": false,  
    "model": "./t5-simplifier"  
}
```

<<<BOLD>>>Output (stdout JSON line):<<<ENDBOLD>>>

```
{  
    "simplified": "Simplified text output",  
    "metrics": {  
        "before": {"word_count": 100, "avg_sentence_length": 15.5, "flesch_reading_ease": 65.2},  
        "after": {...},  
        "change": {...}  
    }  
}
```

<<<BOLD>>>Error response:<<<ENDBOLD>>>

```
{"error": "Error message here"}
```

How to Modify

<<<BOLD>>>Add new request field:<<<ENDBOLD>>>

1. Add to JSON parsing section (~line 70)
2. Pass to <<process_text()<<<ENDFONT>>> call
3. Update Electron <<main.js<<<ENDFONT>>> to send the new field

<<<BOLD>>>Change timeout behavior:<<<ENDBOLD>>>

- Timeout is handled in Electron <<main.js<<<ENDFONT>>> (30 seconds), not in Python

1.3 utils.py - Shared Utilities

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\utils.py<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Common text processing and metrics functions

Functions

Function	Signature	Description	Example Use
<<read_input_file<<<ENDFONT>>>	<<(file_path: str) -> str<<<ENDFONT>>>	Read UTF-8 file, exit on error	CLI input reading
<<split_sentences<<<ENDFONT>>>	<<(text: str) -> list[str]<<<ENDFONT>>>	Split on <<[.!?]<<<ENDFONT>>> while protecting abbreviations	All modes use this
<<get_words<<<ENDFONT>>>	<<(text: str) -> list[str]<<<ENDFONT>>>	Extract words, expand 159 contractions	Metrics calculation
<<count_syllables<<<ENDFONT>>>	<<(word: str) -> int<<<ENDFONT>>>	Heuristic syllable count	Flesch score

<<compute_avg_sentence_length<<<ENDFONT>>>	<<(text: str) -> float<<<ENDFONT>>>	Words per sentence	Metrics
<<compute_flesch_reading_ease<<<ENDFONT>>>	<<(text: str) -> float<<<ENDFONT>>>	Standard Flesch formula	Metrics
<<compute_word_count<<<ENDFONT>>>	<<(text: str) -> int<<<ENDFONT>>>	Word count	Metrics
<<compute_metrics<<<ENDFONT>>>	<<(before: str, after: str) -> dict<<<ENDFONT>>>	Full metrics comparison	API response
<<print_metrics<<<ENDFONT>>>	<<(metrics: dict) -> None<<<ENDFONT>>>	CLI pretty-print table	Debug output

How to Modify

<<<BOLD>>>Add new abbreviation:<<<ENDBOLD>>>

Around line 15-25

```
abbreviations = [
```

```
...
```

```
"newabbr", # Add here
```

```
]
```

<<<BOLD>>>Add new contraction:<<<ENDBOLD>>>

Around line 40-170

```
contractions = {  
    ...  
    "newcontraction": "expanded form",  
}  
  
<<<BOLD>>>Add new metric:<<<ENDBOLD>>>  
  
def compute_new_metric(text: str) -> float:  
    # Your implementation  
    pass  
  
def compute_metrics(before, after):  
    return {  
        ...  
        "new_metric": {  
            "before": compute_new_metric(before),  
            "after": compute_new_metric(after),  
            "change": ...  
        }  
    }
```

1.4 dyslexia_mode.py

```
<<<BOLD>>>Location:<<<ENDBOLD>>>                                <<<FONT                                name="Courier"  
color="#2563EB">F:\chat_editor_mp\dyslexia_mode.py<<<ENDFONT>>>  
  
<<<BOLD>>>Purpose:<<<ENDBOLD>>> Dyslexia-specific post-processing
```

Functions

Function	Signature	Description
<<_split_on_conjunctions<<<ENDFONT>>>	<<(sentence: str) -> list[str]<<<ENDFONT>>>	Split on 32 conjunctions and clause markers
<<_ensure_sentence_end<<<ENDFONT>>>	<<(sentence: str) -> str<<<ENDFONT>>>	Add '.' if missing

<<_capitalize_first<<<ENDFONT>>>	<<(sentence: str) -> str<<<ENDFONT>>>	Capitalize first letter
<<_hyphenate_word<<<ENDFONT>>>	<<(word: str) -> str<<<ENDFONT>>>	VC-CV hyphenation for words >6 chars
<<_hyphenate_text<<<ENDFONT>>>	<<(text: str) -> str<<<ENDFONT>>>	Apply hyphenation preserving punctuation
<<format_for_dyslexia<<<ENDFONT>>>	<<(text, split_sentences_func, use_hyphenation) -> str<<<ENDFONT>>>	Main formatter

Conjunction List (32 items)

<<and, but, or, nor, for, so, yet, which, that, because, although, though, since, unless, until, when, while, where, how, however, moreover, furthermore, nevertheless, nonetheless, consequently, therefore, thus, hence, often, also<<<ENDFONT>>>

How to Modify

<<<BOLD>>>Add new conjunction:<<<ENDBOLD>>>

Line ~6

```
pattern = r'\s<<<ITALIC>>>,?\s<<<ENDITALIC>>>\b(and|but|or|\.\. |newword)\b\s*'
```

<<<BOLD>>>Change minimum fragment filter:<<<ENDBOLD>>>

Line ~92

```
if len(part.split()) < 3: # Change this threshold
```

```
    continue
```

<<<BOLD>>>Adjust hyphenation rules:<<<ENDBOLD>>>

Line ~50-60

Rule 1: VC-CV pattern

```
pattern1 = r'([aeiou][^aeiou])([aeiou][aeiou])'
```

Rule 2: V-CV pattern

```
pattern2 = r'([aeiou])([^\aeiou][aeiou])'
```

1.5 adhd_mode.py

```
<<<BOLD>>>Location:<<<ENDBOLD>>> <<<FONT name="Courier" color="#2563EB">F:\chat_editor_mp\adhd_mode.py<<<ENDFONT>>>
```

```
<<<BOLD>>>Purpose:<<<ENDBOLD>>> ADHD-specific focus formatting
```

Functions

Function	Signature	Description
<<_bold_first_noun<<<ENDFONT>>>	<<(sentence: str) -> str<<<ENDFONT>>>	Bold first significant content word
<<format_for_adhd<<<ENDFONT>>>	<<(text: str) -> str<<<ENDFONT>>>	Main formatter with <<[i/N]<<<ENDFONT>>> markers

Skip-Words List (27 items)

Articles, prepositions, be-verbs, auxiliaries, common adverbs

```
<<<BOLD>>>Heuristic logic:<<<ENDBOLD>>>
```

1. Split sentence into words
2. Skip: articles, prepositions, auxiliaries, pronouns
3. Skip: words ending in <<ing<<<ENDFONT>>>, <<ed<<<ENDFONT>>>, <<es<<<ENDFONT>>>, <<s<<<ENDFONT>>> (verb indicators)
4. Bold first remaining word with length > 2
5. Wrap in <<<<<BOLD>>>word<<<ENDBOLD>>><<<ENDFONT>>>

How to Modify

```
<<<BOLD>>>Add to skip list:<<<ENDBOLD>>>
```


Line ~13-26

```
skip_words = {  
...  
"newword",  
}  
<<<BOLD>>>Change verb-ending detection:<<<ENDBOLD>>>
```

Line ~40-41

if word.endswith(('ing', 'ed', 'es', 's')) and len(word) > 3:

```
# Modify this logic
```

1.6 autism_mode.py

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\autism_mode.py<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Autism-specific idiom and jargon replacement

Functions

Function	Signature	Description
<<load_idiom_map<<<ENDFONT>>>	<<() -> dict<<<ENDFONT>>>	Load from JSON or use fallback
<<load_jargon_map<<<ENDFONT>>>	<<() -> dict<<<ENDFONT>>>	Load from JSON or use fallback
<<_replace_idioms<<<ENDFONT>>>	<<(text: str) -> str<<<ENDFONT>>>	Case-insensitive replacement
<<_replace_jargon<<<ENDFONT>>>	<<(text: str) -> str<<<ENDFONT>>>	Case-sensitive for acronyms, case-insensitive for phrases
<<format_for_autism<<<ENDFONT>>>	<<(text: str) -> str<<<ENDFONT>>>	Main formatter

Current Idiom Count: 20

Current Jargon Count: 20

How to Modify

<<<BOLD>>>Add new idiom:<<<ENDBOLD>>>

Edit <<idiom_map.json<<<ENDFONT>>>:

```
{
```

```
...  
"new idiom": "literal meaning"  
}  
  
<<<BOLD>>>Add new jargon:<<<ENDBOLD>>>  
  
Edit <<<FONT name="Courier" color="#2563EB">jargon_map.json<<<ENDFONT>>>:
```

```
{  
...  
"NEWACRONYM": "plain language expansion"  
}
```

<<<BOLD>>>Note:<<<ENDBOLD>>> Acronyms (uppercase) are matched case-sensitively. Phrases are case-insensitive.

4. Electron Main Process

4.1 main.js

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\main.js<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Electron main process - window creation, Python process management, IPC handling

Key Components

1. <<<BOLD>>>Window creation<<<ENDBOLD>>> (<<createWindow()<<<ENDFONT>>>)

- 1200x800 default size

- Loads Vite dev server or built files

2. <<<BOLD>>>Python process spawning<<<ENDBOLD>>>

- Spawns <<simplify_server.py<<<ENDFONT>>> as child process

- Handles stdout/stderr logging

- Restarts on macOS activation

- Kills on app quit

3. <<<BOLD>>>IPC Handler<<<ENDBOLD>>> (<<ipcMain.handle('simplify', ...)<<<ENDFONT>>>)

- Receives payload from renderer

- Sends JSON + newline to Python stdin

- Buffers stdout to accumulate complete JSON

- 30-second timeout

- Returns parsed response or error

How to Modify

<<<BOLD>>>Change window size:<<<ENDBOLD>>>

// Line ~20

```
const mainWindow = new BrowserWindow({  
  width: 1200, // Change here  
  height: 800, // Change here  
  ...  
});
```

<<<BOLD>>>Adjust timeout:<<<ENDBOLD>>>

// Line ~80

```
const timeout = setTimeout(() => {  
  reject(new Error('Request timeout after 30s'));  
}, 30000); // Change milliseconds here
```

4.2 preload.js

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\preload.js<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Context bridge exposing safe IPC to render

Exposed API

window.electronAPI.simplify(payload)

How to Modify

<<<BOLD>>>Add new IPC channel:<<<ENDBOLD>>>

// Add to contextBridge.exposeInMainWorld

```
newFunction: (arg) => ipcRenderer.invoke('new-channel', arg)
```

Then add handler in <<main.js<<<ENDFONT>>>:

```
ipcMain.handle('new-channel', async (event, arg) => {  
  // Handle new channel
```

```
});
```

5. React UI Layer

5.1 AppContext.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>>

<<F:\chat_editor_mp\electron-app\src\context\AppContext.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Global application state management

State Variables

State	Type	Default	Description
<<inputText<<<ENDFONT>>>	string	<<""<<<ENDFONT>>>	User's input text
<<outputText<<<ENDFONT>>>	string	<<""<<<ENDFONT>>>	Simplified output
<<isLoading<<<ENDFONT>>>	boolean	<<false<<<ENDFONT>>>	Processing state
<<mode<<<ENDFONT>>>	string	<<"dyslexia"<<<ENDFONT>>>	Current mode
<<metrics<<<ENDFONT>>>	object	<<{before: {}, after: {}, change: {}}<<<ENDFONT>>>	Readability metrics
<<focusMode<<<ENDFONT>>>	boolean	<<false<<<ENDFONT>>>	(Unused placeholder)
<<focusSentenceIndex<<<ENDFONT>>>	number	<<0<<<ENDFONT>>>	ADHD mode active sentence
<<sentences<<<ENDFONT>>>	array	<<[]<<<ENDFONT>>>	Split sentences for navigation

<<settingsOpen<<<ENDFONT>>>	boolean	<<false<<<ENDFONT>>>	Settings panel visibility
<<settings<<<ENDFONT>>>	object	See below	User preferences

Settings Object

```
settings: {
  theme: "light", // Handled by ThemeContext
  fontFamily: "lexend", // "lexend" | "opendyslexic" | "merriweather" | "mono"
  fontSize: 18, // 14-28 pixels
  lineSpacing: "standard", // "standard" | "relaxed" | "wide"
  cogFocusMode: true, // (Unused)
  reduceMotion: false, // Disable animations
  useHyphenation: false, // Enable dyslexia hyphenation
}
```

How to Modify

<<<BOLD>>>Add new setting:<<<ENDBOLD>>>

1. Add to <<settings<<<ENDFONT>>> initial state
2. Add side effect in <<useEffect<<<ENDFONT>>> if needed
3. Add UI control in <<SettingsPanel.jsx<<<ENDFONT>>>
4. Apply in relevant component

5.2 ThemeContext.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\context\ThemeContext.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Theme switching (light/dark/high-contrast)

Themes

- <<light<<<ENDFONT>>> - Default light theme
- <<dark<<<ENDFONT>>> - Dark mode
- <<high-contrast<<<ENDFONT>>> - Yellow on black (WCAG AAA)

How to Modify

<<<BOLD>>>Add new theme:<<<ENDBOLD>>>

1. Add CSS variables in <<index.css<<<ENDFONT>>> under new class (e.g., <<html.sepia<<<ENDFONT>>>)
2. Add option in <<SettingsPanel.jsx<<<ENDFONT>>>
3. Add case in <<ThemeContext.jsx<<<ENDFONT>>> effect

5.3 Header.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\layout\Header.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> App header with logo, mode selector, simplify button, settings toggle

Used State

- <<mode<<<ENDFONT>>>, <<setMode<<<ENDFONT>>> - Mode dropdown
- <<isLoading<<<ENDFONT>>> - Button disabled state
- <<handleSimplify<<<ENDFONT>>> - Button action
- <<setSettingsOpen<<<ENDFONT>>> - Settings button

How to Modify

<<<BOLD>>>Change mode options:<<<ENDBOLD>>>

Edit the dropdown options (~line 50-65)

<<<BOLD>>>Change button text/style:<<<ENDBOLD>>>

Modify the "Simplify Text" button (~line 70)

5.4 SettingsPanel.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\layout\SettingsPanel.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Accessibility settings modal

Sections

1. <<<BOLD>>>Display & Theme<<<ENDBOLD>>> - Light/Dark/High-Contrast
2. <<<BOLD>>>Typography<<<ENDBOLD>>> - Font family, size, hyphenation
3. <<<BOLD>>>Reading Assistance<<<ENDBOLD>>> - Line spacing, reduce motion

How to Modify

<<<BOLD>>>Add new setting:<<<ENDBOLD>>>

1. Add UI control in appropriate section
2. Use <<updateSetting('newKey', value)<<<ENDFONT>>> to save
3. Apply effect in <<AppContext.jsx<<<ENDFONT>>> if needed

5.5 InputPanel.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\editor\InputPanel.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Text input area for source text

Used State

- <<inputText<<<ENDFONT>>>, <<setInputText<<<ENDFONT>>> - Text content
- <<mode<<<ENDFONT>>> - Conditional styling for dyslexia

Features

- Clear button (trash icon)
- Character/word count display
- Optional <<dyslexia-text<<<ENDFONT>>> CSS class

5.6 OutputPanel.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\editor\OutputPanel.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Container for mode-specific output renderers

Used State

- <<outputText<<<ENDFONT>>> - Simplified content
- <<isLoading<<<ENDFONT>>> - Loading overlay
- <<mode<<<ENDFONT>>> - Component selection

Features

- Copy to clipboard
- Print button
- Loading overlay with message
- Privacy badge
- Mode-based component routing

5.7 DyslexiaOutput.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\modes\DyslexiaOutput.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Dyslexia-friendly output with text-to-speech

Features

- Web Speech API integration

- Paragraph highlighting synced with speech
- Auto-scrolling (respects reduce-motion)
- Listen/Stop toggle

How to Modify

<<<BOLD>>>Change speech rate:<<<ENDBOLD>>>

// Add around line 30

```
utterance.rate = 0.9; // 0.5 to 2.0
```

<<<BOLD>>>Change voice:<<<ENDBOLD>>>

```
utterance.voice = speechSynthesis.getVoices().find(v => v.name === 'VoiceName');
```

5.8 ADHDFocusMode.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\modes\ADHDFocusMode.jsx<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Cognitive focus navigator - one sentence at a time

Features

- <<parseLine()<<<ENDFONT>>> - Extract <<[i/N]<<<ENDFONT>>> markers and bold text
- Keyboard navigation (arrow keys)
- Click to select sentence
- Previous/Next buttons
- Dimmed context for non-active sentences
- Auto-scroll (respects reduce-motion)

How to Modify

<<<BOLD>>>Change key bindings:<<<ENDBOLD>>>

// Line ~35

```
if (e.key === 'ArrowDown' || e.key === 'ArrowRight') { ... }
```

<<<BOLD>>>Change dimming level:<<<ENDBOLD>>>

/<<<ITALIC>>> index.css <<<ENDITALIC>>>/

```
.dimmed-text {
```

```
opacity: 0.3; /<<<ITALIC>>> Change this <<<ENDITALIC>>>/
```

```
}
```

5.9 AutismOutput.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\modes\AutismOutput.jsx<<<ENDFONT>>>
<<<BOLD>>>Purpose:<<<ENDBOLD>>> Clean paragraph output with bolded replacements

Features

- <<renderBold()<<<ENDFONT>>> - Split on <<**<<<ENDFONT>>> markers
- Clean typography
- Centered layout

How to Modify

<<<BOLD>>>Change replacement highlight color:<<<ENDBOLD>>>

// Line ~10

```
<strong className="text-[var(--color-primary)] font-bold">
```

// Change to different CSS variable or class

5.10 MetricsBar.jsx

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\components\layout\MetricsBar.jsx<<<ENDFONT>>>
<<<BOLD>>>Purpose:<<<ENDBOLD>>> Display readability metrics before/after

Metrics Shown

- Word count
- Average sentence length
- Flesch Reading Ease
- Change badge (green if improved)

5.11 index.css

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\electron-app\src\index.css<<<ENDFONT>>>
<<<BOLD>>>Purpose:<<<ENDBOLD>>> Global styles with CSS variables and Tailwind

Key CSS Variables

```
:root {  
  --color-bg: #f6f6f8;  
  --color-surface: #ffffff;  
  --color-primary: #3b82f6;  
  --color-text: #1f2937;
```

```
--color-muted: #6b7280;
--font-size-body: 18px;
```

```
}
```

Theme Classes

- <<html.light<<<ENDFONT>>> (default)
- <<html.dark<<<ENDFONT>>>
- <<html.high-contrast<<<ENDFONT>>>

Utility Classes

- <<.dyslexia-text<<<ENDFONT>>> - Optimized spacing for dyslexia
- <<.reduce-motion<<<ENDFONT>>> - Disables animations
- <<.fade-in<<<ENDFONT>>>, <<.slide-in-up<<<ENDFONT>>>, <<.pulse-subtle<<<ENDFONT>>> - Animations
- <<.lexend-font<<<ENDFONT>>>, <<.opendyslexic-font<<<ENDFONT>>>, <<.merriweather-font<<<ENDFONT>>>, <<.mono-font<<<ENDFONT>>>

How to Modify

<<<BOLD>>>Change theme colors:<<<ENDBOLD>>>

Edit the appropriate <<html.dark<<<ENDFONT>>> or <<html.high-contrast<<<ENDFONT>>> section

<<<BOLD>>>Add new animation:<<<ENDBOLD>>>

```
@keyframes new-animation {
  from { ... }
  to { ... }
```

```
}
```

```
.new-animation-class {
  animation: new-animation 0.3s ease-out;
```

```
}
```

6. Data Files

6.1 idiom_map.json

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\idiom_map.json<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Idiom → literal meaning mappings for Autism mode

```
<<<BOLD>>>Format:<<<ENDBOLD>>> <<<FONT name="Courier" color="#2563EB">{"idiom phrase": "literal meaning", ...}<<<ENDFONT>>>
```

<<<BOLD>>>Current Count:<<<ENDBOLD>>> 20 entries

Sample Entries

```
{
  "piece of cake": "easy",
  "break a leg": "good luck",
  "under the weather": "feeling sick"
}
```

6.2 jargon_map.json

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\jargon_map.json<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Jargon → plain language mappings for Autism mode

```
<<<BOLD>>>Format:<<<ENDBOLD>>> <<<FONT name="Courier" color="#2563EB">{"term": "plain language", ...}<<<ENDFONT>>>
```

<<<BOLD>>>Current Count:<<<ENDBOLD>>> 20 entries

Sample Entries

```
{
  "MRI": "magnetic resonance imaging",
  "hypertension": "high blood pressure",
  "efficacy": "how well it works"
}
```

7. Model Artifacts

7.1 t5-simplifier/

<<<BOLD>>>Location:<<<ENDBOLD>>> <<F:\chat_editor_mp\t5-simplifier\<<<ENDFONT>>>

<<<BOLD>>>Purpose:<<<ENDBOLD>>> Fine-tuned T5-small model for text simplification

Files

File	Purpose
------	---------

<<config.json<<<ENDFONT>>>	Model architecture config (hidden_size: 768, layers: 12, heads: 12)
<<generation_config.json<<<ENDFONT>>>	Default generation params (overridden by code)
<<model.safetensors<<<ENDFONT>>>	Trained weights
<<tokenizer.json<<<ENDFONT>>>	SentencePiece tokenizer
<<tokenizer_config.json<<<ENDFONT>>>	Tokenizer settings

Training Details

- Dataset: GEM/wiki_auto_asset_turk (filtered for meaningful simplification)
- Prefix: <<"simplify: "<<<ENDFONT>>>
- Checkpoints: 2500, 5000 steps available

8. Data Flow

Complete Request Flow

1. USER TYPES TEXT

2. InputPanel.jsx → setInputText()

3. USER CLICKS "Simplify Text"

4. AppContext.handleSimplify()

5. window.electronAPI.simplify({text, mode, useHyphenation})

6. preload.js → ipcRenderer.invoke('simplify', payload)

7. main.js → ipcMain.handle('simplify')

8. Send JSON to Python stdin: {"text": "...", "mode": "...", ...}

9. simplify_server.py receives line

10. process_text() called

```
simplify_with_t5() → T5 inference per sentence
```

```
dyslexia_mode.format_for_dyslexia()
```

```
OR adhd_mode.format_for_adhd()
```

```
OR autism_mode.format_for_autism()
```

```
compute_metrics(before, after)
```

11. JSON response: {"simplified": "...", "metrics": {...}}

12. main.js receives stdout, parses JSON

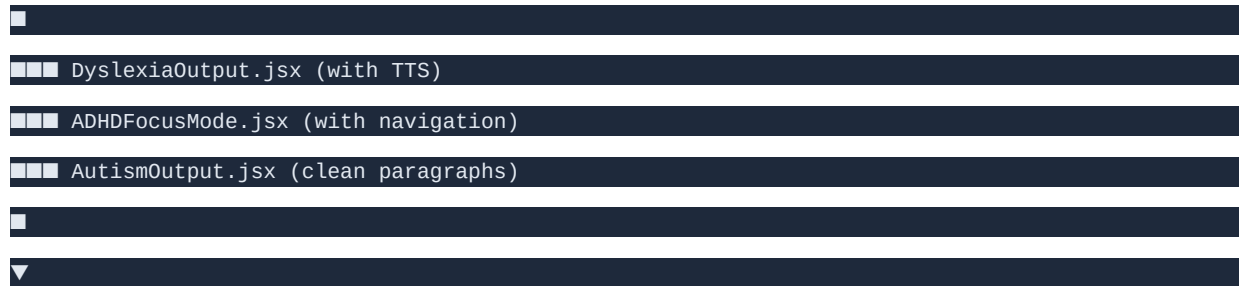
13. AppContext receives result

```
setOutputText(result.simplified)
```

```
setMetrics(result.metrics)
```

```
setSentences(split result)
```

14. OutputPanel.jsx renders mode-specific component



15. USER READS SIMPLIFIED OUTPUT

9. How To Guide

Changing a Button Color

<<<BOLD>>>Example: Change "Simplify Text" button to green<<<ENDBOLD>>>

File: <<Header.jsx<<<ENDFONT>>>

// Find the button (~line 70)

<button

```
onClick={handleSimplify}
```

```
className="... bg-[var(--color-primary)] ..." // Change this
```

>

Change to:

```
className="... bg-green-600 hover:bg-green-700 ..."
```

Adding a New Font

<<<BOLD>>>Step 1:<<<ENDBOLD>>> Add font to <<index.html<<<ENDFONT>>>

```
<link href="https://fonts.googleapis.com/css2?family=NewFont&display=swap" rel="stylesheet">
```

<<<BOLD>>>Step 2:<<<ENDBOLD>>> Add CSS class in <<index.css<<<ENDFONT>>>

```
.newfont-font {
```

```
font-family: 'NewFont', sans-serif;
```

```
}
```

<<<BOLD>>>Step 3:<<<ENDBOLD>>> Add to <<SettingsPanel.jsx<<<ENDFONT>>> dropdown (~line 115)

```
<option value="newfont">NewFont (Custom)</option>
```

<<<BOLD>>>Step 4:<<<ENDBOLD>>> Handle in <<AppContext.jsx<<<ENDFONT>>> useEffect if needed

Adding a New Mode

<<<BOLD>>>Step 1:<<<ENDBOLD>>> Create backend formatter

new_mode.py

```
def format_for_new_mode(text: str) -> str:
```

```
    # Your processing logic
```

```
    return processed_text
```

```
<<<BOLD>>>Step      2:<<<ENDBOLD>>>      Wire      into      <<<FONT      name="Courier"
color="#2563EB">simplify.py<<<ENDFONT>>>
```

In process_text()

```
elif mode == "new_mode":  
  
    return format_for_new_mode(simplified)  
  
<<<BOLD>>>Step      3:<<<ENDBOLD>>>      Wire      into      <<<FONT      name="Courier"  
color="#2563EB">simplify_server.py<<<ENDFONT>>>  
  
from new_mode import format_for_new_mode  
  
<<<BOLD>>>Step 4:<<<ENDBOLD>>> Create React component  
  
// NewModeOutput.jsx  
  
export default function NewModeOutput() {  
    const { outputText } = useApp();  
    return <div>{outputText}</div>;  
}  
  
<<<BOLD>>>Step      5:<<<ENDBOLD>>>      Route      in      <<<FONT      name="Courier"  
color="#2563EB">OutputPanel.jsx<<<ENDFONT>>>  
  
import NewModeOutput from './modes/NewModeOutput';  
  
// ...  
  
{mode === 'new_mode' && <NewModeOutput />}  
  
<<<BOLD>>>Step      6:<<<ENDBOLD>>>      Add      to      <<<FONT      name="Courier"  
color="#2563EB">Header.jsx<<<ENDFONT>>> dropdown
```

Adjusting T5 Model Parameters

```
<<<BOLD>>>File:<<<ENDBOLD>>>      <<<FONT      name="Courier"  
color="#2563EB">simplify.py<<<ENDFONT>>> (~line 115)  
  
outputs = model.generate(  
    inputs["input_ids"],  
    max_length=128, # Max output tokens  
    num_beams=4, # Beam search width  
    length_penalty=0.8, # < 1 = shorter outputs  
    no_repeat_ngram_size=3, # Prevent repetition  
    early_stopping=True, # Stop when all beams finish  
)
```

<<<BOLD>>>To make outputs shorter:<<<ENDBOLD>>> Lower <<max_length<<<ENDFONT>>> or <<length_penalty<<<ENDFONT>>>

<<<BOLD>>>To make outputs longer:<<<ENDBOLD>>> Increase <<max_length<<<ENDFONT>>> or raise <<length_penalty<<<ENDFONT>>> above 1.0

<<<BOLD>>>To improve quality:<<<ENDBOLD>>> Increase <<num_beams<<<ENDFONT>>> (slower)

Expanding the Idiom Dictionary

<<<BOLD>>>File:<<<ENDBOLD>>> <<idiom_map.json<<<ENDFONT>>>

Add new entries following the pattern:

```
{  
  "raining cats and dogs": "raining heavily",  
  "hit the sack": "go to sleep",  
  "your new idiom": "its literal meaning"  
}
```

<<<BOLD>>>Note:<<<ENDBOLD>>> Use lowercase phrases; matching is case-insensitive.

<<<BOLD>>>Note:<<<ENDBOLD>>> Avoid partial idioms that could match incorrectly.

Summary

This codebase implements a complete offline text simplification pipeline with:

1. <<<BOLD>>>Python Backend<<<ENDBOLD>>> - T5 neural simplification + rule-based mode-specific post-processing
2. <<<BOLD>>>Electron Shell<<<ENDBOLD>>> - Desktop app with Python process management
3. <<<BOLD>>>React UI<<<ENDBOLD>>> - Mode-specific output renderers with accessibility features
4. <<<BOLD>>>Multi-modal Support<<<ENDBOLD>>> - Dyslexia, ADHD, and Autism modes
5. <<<BOLD>>>Full Privacy<<<ENDBOLD>>> - Everything runs locally, no data leaves the device

For any modifications, identify the relevant layer (backend, Electron main, React UI) and follow the patterns established in the existing code.

<<<ITALIC>>>Document created: 2026-03-24<<<ENDITALIC>>>

<<<ITALIC>>>Last updated: 2026-03-24<<<ENDITALIC>>>